SP-14 Green Chess AI
Development Document

4850-01
Spring 2026
Professor Perry
February 8, 2026

SP-14 Green Chess AI

Table of Contents

1. TECHNICAL DETAIL SUMMARY 3

- 1.1 CHESSBOARD 3
- 1.2 CHESS GAME ENGINE 3
- 1.3 PIECES – RULES 4
- 1.4 MODES OF PLAY 4
- 1.5 AI STRUCTURE AND LOGIC 5

2. DATABASE CONNECTION 5

3. DEVELOPMENT 5

1. Technical Detail Summary

– *1.1 Chessboard*

Dimensions.py has fixed board dimension values, 100x100 pixels a square.
The board is initialized as a 2D array to represent each square. It then places pictures of pieces in their designated starting positions.

I/O features:

- Square is highlighted on hover
- Legal moves are shown when dragging initiates

– *1.2 Chess game engine*

Main.class runs the main game loop. It listens for mouse and keyboard events, redraws the screen, and runs the game logic once per frame.

Jeu.class tracks the current status of the game. It stores the current player in **next_player**. It knows about the board state, it passes events to Drag if a piece is being dragged, it displays valid moves. It highlights the last move made. When a piece is successfully moved, it calls **next_turn()** to change whose turn it is.

Board.class actually implements all of the movement logic. It stores the board itself in **self.squares**. It positions all the pieces at game start. It knows how to calculate legal moves for each piece. It can validate if a move is legal or not. It updates pieces' positions on the board. It takes care of removing captured pieces from the board. It deals with pawn promotion, castling, and en passant. It remembers the last move that was made.

Piece.class, Pawn.class, Knight.class, Bishop.class, Rook.class, Queen.class, and King.class represent each of the chess pieces. Each piece knows its color, whether it has moved yet or not, and a list of its current legal moves. The engine calculates those legal moves.

Move.class defines a move as having a starting square and an ending square.

Drag.class deals with player interaction when a piece is selected and dragged around the screen.

When the program runs:

Main.mainloop() calls its **run()** method, which runs infinitely inside of a while True:. While that loop is running, the game board is alive. Inside of that loop, the board is drawn, pieces are drawn, valid moves are highlighted, mouse movements are checked, and mouse button clicks/releases are checked.

If the player clicks on a square, **board.square_select(row, col)** determines if there is a piece on that square. If there is, and that piece belongs to the player whose turn it is, **board.moves_calc(piece, row, col, bool=True)** calculates all the piece's legal moves.

If the player clicks and drags the piece to a different square, **Move(move_row, move_col, piece.row, piece.col)** is created, identifying the piece's starting square and ending square. **board.valid_move(piece, move)** is then called to determine if the ending square is in the piece's list of legal moves.

If the move is legal, **board.move(piece, move)** will move the piece. It edits the board so that the piece is removed from its old square and added to its new square. If necessary, it will also remove captured pieces, and deal with en passant, pawn promotion, or castling. Finally, it will set **last_move** to the moved moved, erase the old valid moves, set the piece moved attribute to True, and **game.next_turn()** will set **next_player** to the other player.

– *1.3 Pieces – rules*

- Pawn
    - Has a property direction to differentiate between white and black moves. An if statement is included for the first pawn move being able to move 2 squares. Conditions are included for an opponent piece being diagonally in front of the pawn, as well as being blocked from the front.
    - When a pawn is pushed two spaces, the space behind the pushed pawn becomes capturable for the next move. This enables the player to capture the pawn en passant, if their own pawn is available to execute the move.
- Rook, Bishop, and Queen all use the **straight_moves** method, and the **straight_moves** method iterates for all possible squares in the parameterized direction. It breaks and stops iterating once the edge of the board is reached or if there is a piece in the way (breaks on the square of the opponent piece, breaks on the square before your own piece).
    - Rook utilizes the straight moves along the rows and columns.
    - Bishop utilizes the straight moves along the diagonals.
    - Queen utilizes the straight moves of all eight directions.
- Knight has eight possible moves to make. For each possible move it checks if the square is a legal square to land on.
- King has the ability to move one square in any direction
    - Since each piece has a ‘moved’ property, an unmoved king and the unmoved rook on it’s castling side has the ability to castle by altering the column position of both pieces.

– *1.4 Modes of play*

- PvP is implemented as a local PvP; in other words, 2 people will sit at one computer and alternate taking moves

- PvAI will have an algorithm-based program playing against the player.

– *1.5 AI structure and logic*

- The evaluation algorithm is a negamax with alpha beta pruning.
- Pieces are assigned a constant value that corresponds to how valuable the piece is.
- A constant piece table is assigned to calculate the value of the piece's position on the board.
- Evaluation of the position is calculated by the value of the piece with it's corresponding position in it's table matrix. White pieces add to the evaluation while black pieces subtract from the evaluation.
- Negamax recursively searches the best position while ignoring the worst positions. The best score of the position is kept as the alpha value, and any position that is worse than the alpha value gets beta pruned.
    - When possible moves are searched, the order in which they will be searched in is checks, captures, and attacks since those moves have a higher chance of producing a desirable outcome.
    - The highest four resulting evaluations are explored, leading to more possible better evaluations after making an initial 'poor' move.

2. Database Connection

Not applicable

3. Development

During the development our team was running Python 3.13, but the program is likely to also work with relatively recent older versions.
Pygame is an essential module which is required to run the program.

Installation:
git clone https://github.com/OurChessAi/OurChessProject/
cd SP14-ChessAIGreen/SRC
python main.py